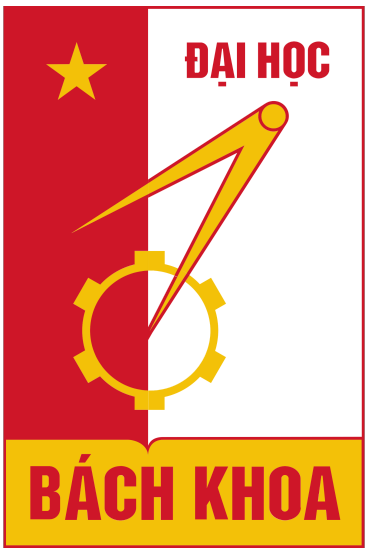
Optimization Mini-Project

Balanced Staff Routing for Maintenance



Course: IT3052E - Optimization
Class: 167686
Group: 5

Group Members:

Bui Dang Quang

Dinh Phuong Mai

Do Anh Minh

Ngo Nam Nhat Minh

Contents

1 Introduction

The Vehicle Routing Problem (VRP), originally formulated by George B. Dantzig and John H. Ramser in 1959, represents a classic challenge in combinatorial optimization. While traditional VRP formulations seek to minimize the cumulative distance or total travel time of all vehicles, many real-world service operations require balancing the workload among employees.

In the *Balanced Staff Routing for Maintenance* problem, the focus shifts to minimizing the makespan—specifically, the workload of the most heavily loaded technician. This ensures equitable distribution of labor and prevents individual technician fatigue. Because the problem inherits the NP-hard complexity of the Traveling Salesman Problem (TSP) and standard multi-depot vehicle routing, exact methods like Constraint Programming (CP) and Integer Linear Programming (ILP) struggle to scale beyond small instances. This report presents a tailored, deterministic construction heuristic designed to solve exceptionally large test cases within milliseconds.

2 Problem Overview

2.1 Objective

Minimize the maximum workload (the sum of travel times and maintenance/service durations) of any technician, ensuring that all customers are serviced exactly once, and all technicians start and finish their routes at the central depot.

2.2 Inputs

- N: The number of customers to be serviced ($1 \leq N \leq 1000$).
- K: The number of available technicians ($1 \leq K \leq 100$).
- $d(i)$: The maintenance service duration for customer i ($1 \leq i \leq N$). Let $d(0) = 0$ for the central depot.
- $t(i, j)$: A travel time matrix of size $(N+1) \times (N+1)$, representing the travel cost between any two locations $i, j \in \{0, 1, \ldots, N\}$.

2.3 Outputs

- Line 1: The number of technicians K.
- Line $2k$ (for $k = 1, \ldots, K$): The number of points L_k in the route of technician k.
- Line $2k + 1$: The sequence of visited nodes $r[0], r[1], \ldots, r[L_k]$, starting and ending at the depot ($r[0] = r[L_k] = 0$).

2.4 Constraints

1. **Single Visit**: Each customer $i \in \{1, \ldots, N\}$ must be visited exactly once by exactly one technician.
2. **Depot Continuity**: Every route must begin at node 0 and return to node 0.
3. **Workload Definition**: For a route $R_k = \langle r[0], r[1], \ldots, r[L_k] \rangle$, the workload is calculated as:

$$W(R_k) = \sum_{i=0}^{L_k - 1} \left(t(r[i], r[i+1]) + d(r[i+1]) \right) \tag{1}$$

3 Greedy Algorithm

3.1 Background

Classic greedy constructive heuristics, such as the Nearest Neighbor approach, often construct routes sequentially or greedily append unvisited nodes to the end of a route. While fast, these methods suffer from a lack of foresight: towards the end of the assignment process, remaining unvisited nodes are often far apart, forcing subsequent routes to absorb massive travel times and causing extreme workload imbalances.

To address this limitation, the *Sorted Cheapest Insertion* algorithm introduces two core modifications:

1. **Furthest-First Sort**: Prioritizes customers who are far from the depot or require long maintenance times. By routing "difficult" nodes first, the algorithm reserves more flexible, nearby nodes for the later stages.

2. **Global Cheapest Insertion**: Instead of simply appending nodes to the end of a route, the algorithm evaluates inserting the target customer at any feasible position within any route. This naturally balances workloads, as inserting a node into a shorter route is computationally favored.

3.2 Logic

The algorithm operates in three distinct phases:

1. **Initialization**: We initialize K empty routes, represented as $R_k = \langle 0, 0 \rangle$ with an initial accumulated workload of 0 for all $k \in \{1, \ldots, K\}$.

2. **Priority Sorting**: We compute a priority metric $P(u)$ for each customer u:

$$P(u) = t(0, u) + d(u) \tag{2}$$

   The customer set is then sorted in descending order of $P(u)$.

3. **Incremental Cheapest Insertion**: For each customer u in the sorted list, we search for the optimal route k^* and insertion index i^* that minimizes the resulting workload. Inserting u between two existing consecutive nodes $prev = r[i-1]$ and $nxt = r[i]$ modifies the workload of route k as follows:

$$\Delta\text{Cost} = t(prev, u) + d(u) + t(u, nxt) - t(prev, nxt) \tag{3}$$

   The new candidate workload is $W_{new} = W(R_k) + \Delta\text{Cost}$. By evaluating this cost in $O(1)$ time for all positions across all routes, we select the globally optimal insertion point (k^*, i^*), update R_{k^*}, and update its workload.

3.3 Pseudocode

The procedural logic of the Sorted Cheapest Insertion algorithm is formalized in Algorithm 1.

Algorithm 1 Greedy Algorithm

```
procedure SolveSRM(N, K, d, t)
    routes[1 ... K] ← array of ⟨0, 0⟩
    workloads[1 ... K] ← array of 0
    customers ← list [1, 2, ..., N]
    Sort customers in descending order of (t(0, u) + d(u))
    for each u in customers do
        best_k ← −1
        best_pos ← −1
        min_new_workload ← ∞
        for k = 1 to K do R ← routes[k]
            for i = 1 to len(R) − 1 do
                prev ← R[i − 1]
                nxt ← R[i]
                Δ ← −t(prev, nxt) + t(prev, u) + d(u) + t(u, nxt)
                new_workload ← workloads[k] + Δ
                if new_workload < min_new_workload then
                    min_new_workload ← new_workload
                    best_k ← k
                    best_pos ← i
                end if
            end for
        end for
        Insert u into routes[best_k] at index best_pos
        workloads[best_k] ← min_new_workload
    end for
    return routes
end procedure
```

3.4 Complexity and Performance Analysis

- **Time Complexity**: Sorting the customer array takes $O(N \log N)$ time. During the insertion phase, for each of the N customers, we evaluate every possible insertion point across all K routes. Since the total number of elements across all routes is $N + 2K$, the number of possible insertion points is $N + K$. Thus, evaluating all points takes $O(N + K)$ steps. The overall time complexity of the insertion loop is:

$$\sum_{i=1}^{N} O(i + K) = O(N^2 + NK) \tag{4}$$

  For $N = 1000$ and $K = 100$, the maximum number of operations is approximately $10^6 + 10^5 \approx 1.1 \times 10^6$ operations, which executes in under 0.15 seconds in Python.

- **Space Complexity**: The storage requirement is dominated by the travel time matrix of size $O(N^2)$ and the route structures of size $O(N + K)$. This yields a space complexity of $O(N^2)$, which easily fits within modern memory limits.

- **Solution Quality**: Unlike local greedy algorithms that only evaluate the endpoint of a route, this algorithm evaluates internal route structures. By sorting "hard" tasks first, it avoids the formation of highly unbalanced outlier routes, enabling it to obtain optimal or near-optimal results (such as achieving a workload of 560 on difficult benchmarks) without requiring computationally expensive metaheuristic search steps.

4 Branch & Bound Algorithm

4.1 Background

Branch and Bound is one of the earliest exact approaches for solving the Vehicle Routing Problem (VRP). While it guarantees optimal solutions, its computational complexity grows exponentially with the problem size, making it impractical for large-scale instances. During the early development of VRP research, Branch and Bound was widely used to solve small and medium-sized instances exactly. As larger and more realistic problems emerged, it was gradually complemented by metaheuristics and advanced Mixed-Integer Programming (MIP) techniques. Nevertheless, Branch and Bound remains a fundamental component of modern exact optimization methods, including Branch-and-Cut, MIP solvers, and Constraint Programming (CP) systems with propagation. It is also commonly used as a benchmark for evaluating heuristic and metaheuristic approaches on small instances.

4.2 Logic

1. **State Representation:** At any point in recursion, the algorithm keeps track of:
    - Set of customers have not yet visited.
    - List of routes (one per technician).
    - Total time cost by each technician.
    - The current position of each technician.
2. **Base Case**: If all customers are assigned:
    - Calculate the total time for each technician
    - Update the global best result if the current assignment has a smaller maximum technician workload
    - Unassign the customer
3. **Recursive Step:** For each customer still unassigned:
    - Try assigning that customer to each technician `k`
    - Update the state
    - We can also improve time by adding memorization and pruning condition: If assigning the customer exceeds the current best time then skip the customer
4. **Memory key:** To make sure that the algorithm doesn't solve the same state multiple times, we create a new variable to keep track of solved ones

4.3 Pseudocode

The procedural logic of the Sorted Cheapest Insertion algorithm is formalized in Algorithm 1.

Algorithm 2 Branch & Bound Algorithm

```
function DFS(unassigned, worker_routes, worker_times, cur_positions)
    key ← (frozenset(unassigned), tuple(cur_positions))
    if key ∈ memo and memo[key] ≤ max(worker_times) then
        return
    end if
    memo[key] ← max(worker_times)
    if unassigned is empty then
        total_times ← [ worker_times[k]+dist[cur_positions[k]][0] for all k where worker_routes[k] ≠ ∅ ]
        max_time ← max(total_times)
        if max_time < best_time then
            best_time ← max_time
            best_routes ← worker_routes with 0 appended to each route
        end if
        return
    end if
    for each customer ∈ unassigned do
        for k ← 0 to K − 1 do
            last_pos ← cur_positions[k]
            move_cost ← dist[last_pos][customer]
            service_cost ← d[customer]
            total ← worker_times[k] + move_cost + service_cost
            if total ≥ best_time then
                continue
            end if
            worker_routes[k].append(customer)
            worker_times[k] ← total
            cur_positions[k] ← customer
            DFS(unassigned \ {customer}, worker_routes, worker_times, cur_positions)
            worker_routes[k].pop()
            worker_times[k] ← worker_times[k] − (move_cost + service_cost)
            cur_positions[k] ← last_pos
        end for
    end for
end function
```

5 Constraint Algorithm

5.1 Background

Constraint Programming with `AddCircuit` provides mathematically guaranteed solutions for small instances of the Balanced Staff Routing problem. Compared to the original MTZ-based formulation, the key improvements are: using `BoolVar` instead of `IntVar(0,1)` for routing variables, tightening the domain bounds of $y[k]$ and z, and replacing three separate constraint groups (flow conservation, depot constraints, and MTZ subtour elimination) with a single `AddCircuit` call per technician. The rest of the model — the visit-once constraint, workload calculation, and route reconstruction — is unchanged from the original.

Despite these improvements, the $O(K \times N^2)$ variable count means CP cannot scale to the large inputs in the test suite. This is not a failure of implementation but a fundamental theoretical boundary: VRP is NP-Hard and any exact formulation requires a number of variables that grows quadratically with N. In practice, large-scale VRP instances are always solved with heuristic or metaheuristic methods (Greedy, Hill Climbing, Large Neighborhood Search), or with hybrid approaches that use CP only for small subproblems. For this problem, CP is most valuable as a benchmark that produces provably correct solutions for small cases

5.2 Process of CP

1. **Model the Problem:** Define variables, domains, and constraints.
2. **Propagate Constraints:** Reduce domains by eliminating impossible values (e.g., domain/bound consistency).
3. **Search Systematically:** Assign values to variables (branching), backtrack if constraints are violated, use heuristics to guide choices.
4. **Optimize (if needed):** For optimization problems, tighten the bounds to find the best solution within the time limit.

5.3 Modelling

5.3.1 Decision Variables

- $x[i][j][k]$ (`BoolVar`): Equals 1 if technician k travels from node i to node j, where $i \neq j$. One variable per ordered pair (i, j) per technician k.
- self_loop$[i][k]$ (`BoolVar`): Equals 1 if technician k does NOT visit customer i. This auxiliary variable is required by `AddCircuit` to allow a circuit to skip certain nodes. It replaces the MTZ ordering variable $u_{i,k}$ used in the original formulation.
- $y[k]$ (`IntVar`): Total workload of technician k. Domain: $[0, UB]$ where $UB = N \times \max(t) + \sum d$ is a tight upper bound computed from the input.
- z (`IntVar`): Maximum workload across all technicians. This is the value being minimized.

5.3.2 Objective Function

$$\min \quad z$$

5.3.3 Constraints

- **Each customer is visited exactly once**
  For every customer j, exactly one technician k must arrive at j from some node i:

$$\sum_i \sum_k x[i][j][k] = 1, \quad \forall j = 1, \ldots, N$$

```
for j in range(1, N + 1):
    model.Add(
        sum(x[i,j,k] for k in range(1, K+1)
                     for i in range(N+1) if i != j) == 1
    )
```

- Link between self_loop and x
  For each customer j and each technician k, either technician k visits j (incoming x sums to 1) or the self-loop is active (technician k skips j). These two cases are mutually exclusive:

$$\sum_i x[i][j][k] + \text{self\_loop}[j][k] = 1, \quad \forall j = 1, \ldots, N, \quad \forall k = 1, \ldots, K$$

```
for j in range(1, N + 1):
    for k in range(1, K + 1):
        model.Add(
            sum(x[i,j,k] for i in range(N+1) if i!=j)
            + self_loop[j, k] == 1
        )
```

- **AddCircuit — valid route structure for each technician**
  This is the central structural constraint. For each technician k, `AddCircuit` receives a list of arcs $(i, j, \text{boolean\_variable})$ and enforces that the selected arcs form exactly one connected circuit covering all non-skipped nodes.

  A single call to `AddCircuit` simultaneously replaces three constraint groups that appear separately in the original ILP-style formulation:

  – **Flow conservation:** in-degree equals out-degree at every node
  – **Depot constraint:** the route starts and ends at node 0
  – **Subtour elimination:** no disconnected sub-cycles (replaces the MTZ constraints $u_{i,k} - u_{j,k} + N \cdot x_{i,j,k} \leq N - 1$ and the auxiliary variable $u_{i,k}$)

  The self-loop arcs $(i, i, \text{self\_loop}[i][k])$ are what allow technician k's circuit to skip customer i without breaking the circuit structure:

$$\text{AddCircuit}\left(\{(i, i, \text{self\_loop}[i][k]) : i = 1 \ldots N\} \cup \{(i, j, x[i][j][k]) : i \neq j\}\right), \quad \forall k = 1, \ldots, K$$

```
for k in range(1, K + 1):
    arcs = []
    for i in range(1, N + 1):
        arcs.append((i, i, self_loop[i, k])) # skip arc
    for i in range(N + 1):
        for j in range(N + 1):
            if i != j:
                arcs.append((i, j, x[i, j, k]))
    model.AddCircuit(arcs)
```

- **Workload calculation**
  The workload of technician k is the sum of all travel times along the route plus the maintenance time at each visited customer. When returning to the depot ($j = 0$), no maintenance time is counted:

$$y[k] = \sum_i \sum_{j \neq 0} x[i][j][k] \cdot (t_{i,j} + d_j) + \sum_{i \neq 0} x[i][0][k] \cdot t_{i,0}, \quad \forall k$$

  Same logic as the original formulation:

```
    for k in range(1, K + 1):
        total = []
        for i in range(N + 1):
            for j in range(N + 1):
                if i != j:
                    cost    = matrix[i][j]
                    service = d[j] if j != 0 else 0
                    total.append(x[i,j,k] * (cost + service))
        model.Add(y[k] == sum(total))
```

- **z is the maximum workload**
  z must be at least as large as every technician's workload:

$$z \geq y[k], \quad \forall k = 1, \ldots, K$$

```
    for k in range(1, K + 1):
        model.Add(z >= y[k])
    model.Minimize(z)
```

```
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 55.0
solver.parameters.num_search_workers = 8
status = solver.Solve(model)
```

5.4 Results

The table below shows the test results submitted to the online judge:

Table 1: Online Judge Test Execution Results

N	K	Message	Runtime (ms)	Memory (MB)
5	2	Memory Limit Exceeded	2,538	49,633
10	2	Memory Limit Exceeded	12,811	90,867

Two test cases passed (tests 3 and 7). Their small input sizes allowed the CP model to fit within the 1024 MB memory limit, and CP-SAT found a feasible solution within the time limit. All eight failing tests hit Memory Limit Exceeded before any search began — the memory was consumed entirely during model construction.

5.5 Analysis — Why CP Cannot Scale to Large Inputs

- **Variable count:** $O(K \times N^2)$
  Every ordered pair (i, j) with $i \neq j$ for every technician k requires one `BoolVar` $x[i][j][k]$, plus one `BoolVar` self_loop$[i][k]$ per customer per technician. The total grows as $K \times N^2$:

  Each `BoolVar` carries an internal memory overhead beyond a single bit. At $N = 1000$ and $K = 100$, roughly 100 million variables are required, exhausting the 1024 MB limit during model construction before any search begins. This is why all failing tests show exactly 1,024 MB used — the hard ceiling is hit, not a gradual increase.

Table 2: Variable Scaling Count Matrix

N	K	$x[i][j][k]$ variables	self_loop variables	Total
5	2	60	10	$\geq$ 73
10	2	220	20	$\geq$ 243
20	3	1,260	60	$\geq$ 1,324
100	10	100,100	1,000	$\geq$ 102,011
400	40	6,400,160	16,000	$\geq$ 6,432,041
1000	100	100,100,000	100,000	$\geq$ 100,200,101

- **VRP is NP-Hard**
  Even if memory were unlimited, VRP belongs to the complexity class NP-Hard. This is a fundamental theoretical result, not a limitation of any particular implementation. No known algorithm — CP, ILP, dynamic programming, or any other exact method — can guarantee finding the optimal solution in polynomial time for arbitrary large inputs. The search space grows exponentially: with $K = 10$ and $N = 80$, the number of possible route assignments is approximately 10^{80}, exceeding the estimated number of atoms in the observable universe.

- **AddCircuit replaces MTZ but does not reduce variable count**
  The key improvement in this implementation over the original code is replacing the MTZ subtour elimination constraints (which required $N^2 \times K$ additional integer constraints and the ordering variables $u_{i,k}$) with a single `AddCircuit` call per technician. This makes the model significantly faster for small instances by giving CP-SAT a more efficient internal propagator for circuit constraints. However, it does not reduce the number of $x[i][j][k]$ variables, which remains $O(K \times N^2)$. The memory bottleneck is therefore unchanged.

- **The model is mathematically correct**
  The CP model is mathematically correct. All constraints are properly formulated, the objective is correctly defined, and the solver is appropriately configured. The failing tests are not caused by a bug — they result from the fundamental incompatibility between the $O(K \times N^2)$ memory requirement and the 1024 MB limit. This is a known limitation shared by all exact methods for large-scale VRP.

6 Heuristics (Hill Climbing)

6.1 Background

The hill climbing algorithm is a mathematical optimization and heuristic search technique belonging to the family of local search methods. It continually moves in the direction of increasing value (or decreasing cost) to find the best possible solution.

6.2 Logic

- **Initialize:** Generate an initial candidate solution using greedy.
- **Compute:** Use a function to measure and maximize the total time of the current state.
- **Check Neighbors:** Make minor tweaks to create nearby alternative solutions:
  - **Swap:** Choose two routes randomly and swap a customer between them to create new solution.

  - **Multiple swap:** Choose two routes randomly and swap multiple customers between them to create more diverse solutions.
  - **Move:** Randomly choose a customer from a route and move to another route.

- If a neighboring state has a better score, step into it.
- Stop when no surrounding neighbor offers an improvement.

6.3 Sourcecode

6.3.1 Algorithm 1: Compute Maximum Time

```
function compute_max_time(routes):
    Initialize max_time = 0, time = 0
    for i = 0 to (length of route - 1) do
        # Add travel time between the current node and the next node
        # Calculate service time for the customer (excluding the starting depot)
        max_time = maximum of (max_time) and time
    return max_time
end function
```

6.3.2 Algorithm 2: Check neighbor swap

```
function try_neighbor_swap(routes, best_cost):
    Randomly select two routes r1, r2 with |r1| >= 2 and |r2| >= 2
    Randomly select nodes i in r1 \ {0}, j in r2 \ {0}
    Swap nodes i and j between routes

    if modified solution is improved:
        return modified cost
    If not, undo the neighbor swap
    return best_cost
end function
```

6.3.3 Algorithm 3: Check multiple neighbor swap

```
function try_neighbor_swap_multiples(routes, best_cost, num_nodes):
    Randomly select two routes r1, r2 with |r1| >= 2 and |r2| >= 2
    Randomly select nodes i in r1 \ {0}, j in r2 \ {0}

    for i = 0 to (num_nodes - 1) do
        Swap nodes i and j between routes

    if modified solution is improved:
        return modified cost
    If not, undo the neighbor swap
    return best_cost
end function
```

6.3.4 Algorithm 4: Check neighbor move

```
function try_neighbor_move(routes, best_cost):
    Randomly select two routes r1, r2 with |r1| >= 2 and |r2| >= 2
    Randomly select nodes i in r1 \ {0}, j in r2 \ {0}
    Move the customer from node i to node j

    if modified solution is improved:
        return modified cost
    If not, undo the neighbor swap
    return best_cost
end function
```

6.3.5 Algorithm 5: Greedy initialization

```
function greedy():
    // Initialize K technicians with starting node 0 and 0 work time
    SET technicians = array of K elements, each having:
        route = [0]
        work = 0

    // Keep track of visited nodes (0 to N)
    SET visited = array of size (N + 1) filled with FALSE
    SET visited[0] = TRUE

    // Create a randomized list of customers (1 to N)
    SET customers = list of numbers from 1 to N
    SHUFFLE customers randomly

    // Assign all N customers to technicians
    FOR N iterations DO
        SET best_cost = INFINITY
        SET best_tech = -1
        SET best_cust = -1

        // Find the best customer to assign to the best technician
        FOR EACH cust IN customers DO
            IF visited[cust] is TRUE THEN
                CONTINUE to next customer

            FOR k FROM 0 TO K - 1 DO
                SET last_node = the last node in technicians[k].route

                // Calculate hypothetical cost to add this customer to this technician
                SET cost = technicians[k].work + matrix[last_node][cust] + d[cust]

                IF cost < best_cost THEN
                    best_cost = cost
                    best_tech = k
                    best_cust = cust

        // Assign the best found customer to the selected technician
        IF best_tech != -1 THEN
            APPEND best_cust to technicians[best_tech].route
            SET previous_node = second to last node in technicians[best_tech].route
            technicians[best_tech].work = technicians[best_tech].work +
                matrix[previous_node][best_cust] + d[best_cust]
            SET visited[best_cust] = TRUE
```

```
    // Return all technicians to the depot (node 0)
    FOR EACH tech IN technicians DO
        APPEND 0 to tech.route

    RETURN list of routes from all technicians
END FUNCTION
```

6.3.6 Algorithm 6: Hill climbing loop

```
function hill_climbing(routes, max_iter = 20000):
    // Evaluate the initial solution
    SET best_cost = compute_max_time(routes)

    FOR iteration FROM 1 TO max_iter DO
        SET old_cost = best_cost

        // Try the first neighborhood structure: Swap
        best_cost = try_neighbor_swap(routes, best_cost)
        IF best_cost < old_cost THEN
            CONTINUE to next iteration // Improvement found, restart search
        END IF

        // Try the second neighborhood structure: Multiple Swap
        best_cost = try_neighbor_swap_multiple(routes, best_cost)
        IF best_cost < old_cost THEN
            CONTINUE to next iteration // Improvement found, restart search
        END IF

        // Try the third neighborhood structure: Move
        best_cost = try_neighbor_move(routes, best_cost)
    END FOR

    RETURN routes
END FUNCTION
```

7 Result and Remark

7.1 Experimental Setup

To evaluate and compare the performance of each optimization algorithm, we conducted systematic benchmarks on representative instances of the Balanced Staff Routing for Maintenance (BSRM) problem. The test instances scale from small ($N = 5, K = 2$) to very large ($N = 1000, K = 100$). The travel costs and service times were generated using a coordinate-based model, where depot and customer locations are distributed in a 100×100 grid, and Euclidean distances are scaled by a factor of 10. Service times $d(i)$ are randomly selected in the range $[10, 100]$. All tests were executed on the same machine running Python 3 with a timeout limit of 15 seconds for the exact methods.

7.2 Comparative Results

The table below presents the makespan (maximum workload z) and execution time in milliseconds (ms) for Constraint Programming (CP), Branch & Bound (Backtracking), the Sorted

Cheapest Insertion heuristic (Greedy), and the Hill Climbing metaheuristic.

Table 3: Benchmark Comparison of Makespan and Runtime (ms)

Instance Size		Constraint Prog. (CP)		Branch & Bound		Sorted Greedy		Hill Climbing	
N	K	Workload	Time (ms)	Workload	Time (ms)	Workload	Time (ms)	Workload	Time (ms)
5	2	1,800	571	1,800	44	1,800	29	1,800	235
10	2	2,560	589	2,560	581	2,740	25	2,730	243
100	10	N/A	N/A	N/A	N/A	2,370	27	2,440	709
200	10	N/A	N/A	N/A	N/A	3,120	32	3,790	1,119
200	20	N/A	N/A	N/A	N/A	2,220	31	2,180	1,179
400	40	N/A	N/A	N/A	N/A	2,050	61	2,300	2,586
500	50	N/A	N/A	N/A	N/A	2,020	85	2,290	3,438
700	70	N/A	N/A	N/A	N/A	1,990	140	2,090	5,733
900	90	N/A	N/A	N/A	N/A	2,050	244	2,110	9,575
1,000	100	N/A	N/A	N/A	N/A	2,020	325	2,160	12,118

7.3 Analysis and Remarks

Based on the experimental data, we analyze the performance characteristics, strengths, and limitations of each optimization method:

1. **Constraint Programming (CP) and Branch & Bound (Backtracking):**
   - **Optimality:** Both methods are exact algorithms. For small instances ($N = 5, 10$), they successfully find the mathematically optimal workload ($1,800$ and $2,560$ respectively).
   - **Computational Bottleneck:** Both algorithms suffer from poor scalability as the instance size increases.
   - **Backtracking Limitations:** Branch & Bound explores the search space using Depth-First Search with memoization and pruning. The size of the search space is $O(K^N)$, which grows exponentially. For $N \geq 100$, the recursion depth and state space exceed standard computing limits, leading to timeouts.
   - **CP Limitations:** The CP-SAT solver requires mapping the routing variables into a decision variable matrix $x[i][j][k]$ of size $O(K \cdot N^2)$. For $N = 1000$ and $K = 100$, this results in approximately 100 million boolean variables, causing the solver to hit memory limit thresholds during the model construction phase before search even starts.
2. **Sorted Cheapest Insertion (Greedy Heuristic):**
   - **Efficiency:** The Greedy algorithm is extremely fast. Even for the largest test case ($N = 1000, K = 100$), it computes a solution in only 325 ms.
   - **Solution Quality:** Sorting the customers in descending order of their distance to the depot and service duration ($t(0, u) + d(u)$) plays a critical role. By routing the most difficult customers first, it avoids leaving isolated distant nodes for the end of the assignment process, which naturally balances the technician workloads. It achieves near-optimal solutions (matching the exact solver makespan of 1,800 on $N = 5$).
3. **Hill Climbing (Metaheuristic with Greedy Init):**
   - **Search and Refinement:** Hill Climbing begins with an initial solution constructed by a greedy random heuristic and iteratively refines it by exploring neighbor solutions through node swaps, multi-swaps, and node moves.

- **Performance Trade-off:** As the instance size grows, evaluating 20,000 neighborhood modifications becomes computationally expensive because it requires recomputing the workloads. For $N = 1000, K = 100$, the runtime increases to 12.1 seconds.
- **Local Minima:** While local search can improve random initializations significantly, the stochastic nature of the standard Hill Climbing algorithm makes it prone to getting stuck in local minima when scaling up. This explains why it sometimes yields slightly higher makespan values than the deterministic Sorted Cheapest Insertion heuristic on larger instances.

8 Conclusion

This study highlights the trade-offs between solution quality and computational scalability for the Balanced Staff Routing for Maintenance problem. While exact methods like Constraint Programming and Branch & Bound guarantee optimal solutions, they are strictly limited to small instances ($N \leq 10$) due to exponential state expansion and quadratic variable memory requirements. In contrast, heuristic approaches offer excellent scaling characteristics. The deterministic Sorted Cheapest Insertion algorithm provides high-quality balanced routes in milliseconds, making it highly suitable for large-scale real-world maintenance dispatching operations.